BRANDME

App Development Plan

A staged development roadmap for building
Toronto's creator discovery platform

React Native • Expo • Supabase • Mapbox
February 2026

Overview

BrandMe is a mobile-first social discovery platform for Toronto's creator community. It connects content creators, brand owners, artists, and anyone building an online presence with one another and with local events. Think LinkedIn for creatives, built specifically for Toronto.

This document outlines a staged build plan designed for use with Claude Code. Each stage produces a testable, demoable milestone. Stages are ordered to maximize early feedback and minimize rework.

Guiding Principles

1. Mock Data First, Real Data Later

All screens use typed mock data until the UI is stable. TypeScript interfaces are defined early and mirror the database schema, so swapping in real Supabase calls later is seamless. The only exception is authentication, which is set up with Supabase from the start because so much of the app depends on knowing who the current user is.

2. Direct Supabase Calls from the Frontend

For standard CRUD operations (reading profiles, fetching events, RSVPs, follows), the app calls Supabase directly using @supabase/supabase-js. Row Level Security (RLS) policies handle authorization at the database level. A FastAPI backend is only introduced later for complex operations like advanced geospatial queries, third-party integrations, or sensitive business logic.

3. One Stage, One Testable Milestone

Each stage ends with a clear checkpoint: something you can tap through on your phone and verify works. No stage depends on the next one being complete.

Tech Stack Summary

Frontend: React Native (TypeScript) with Expo SDK 54 managed workflow. State management via TanStack Query. Navigation via React Navigation (bottom tabs + stack navigators).

Map: react-native-maps with Mapbox for custom styling, dark themes, and creator/event pin clustering. Mapbox's free tier covers 25,000 monthly active users.

Backend & Database: Supabase (PostgreSQL + PostGIS) for auth, database, storage, and real-time subscriptions. FastAPI introduced only when needed for complex server-side logic.

Build & Deploy: Expo EAS for cloud builds targeting iOS (TestFlight) and Android (internal testing).

Analytics: PostHog for product analytics, session replays, and heatmaps. Free tier covers up to 1 million events per month.

STAGE 0

Project Scaffolding & TypeScript Foundation

This stage creates the project skeleton and establishes the conventions every future stage will follow. By the end, you have a running Expo app with tab navigation and a complete type system that mirrors your database schema.

Tasks

#	Task	Details
1	**Initialize Expo project**	Run create-expo-app with the blank TypeScript template. Confirm the app runs on your physical device via Expo Go by scanning the QR code.
2	**Install core dependencies**	React Navigation (bottom tabs + native stack), TanStack Query, @supabase/supabase-js, react-native-maps. Do not configure Supabase yet—just install the package.
3	**Build bottom tab navigator**	Five tabs: Home, Map, Create, My Events, Account. Each tab renders a placeholder screen component showing the tab name. Use icons from @expo/vector-icons.
4	**Create /types directory**	Define TypeScript interfaces for all entities: User, Profile (with 'CREATOR' | 'BUSINESS' union type), Niche, ExternalLink, Event, EventRSVP, Post, Follow. These must mirror the database schema exactly.
5	**Create /mock directory**	Build mock data files that conform to the types: 8–10 sample profiles (mix of creators and businesses across niches), 5–6 events with varied types and dates, a master niche list of 15–20 entries. Export as typed arrays.
6	**Set up project structure**	Establish folder conventions: /src/screens, /src/components, /src/navigation, /src/types, /src/mock, /src/hooks, /src/utils, /src/constants (for colors, spacing, fonts). Add a global theme/colors file.

✓ **END-OF-STAGE CHECKPOINT**

- App launches on your phone via Expo Go
- Five tabs are visible and tappable in the bottom nav bar
- Each tab shows its placeholder screen with the correct title
- All TypeScript types are defined and all mock data files compile without errors
- Project folder structure is established and consistent

STAGE 1

Authentication & Onboarding

This is the only stage where you set up real Supabase infrastructure before the UI is complete. Authentication is foundational—navigation guards, profile ownership, and eventually RLS all depend on knowing who the current user is. You'll build the sign-up/login flow and a multi-step onboarding sequence.

Tasks

#	Task	Details
1	**Create Supabase project**	Set up a new Supabase project. Enable email/password auth. Grab your project URL and anon key. Store them in an environment config file (not hardcoded).
2	**Create database tables**	In the Supabase SQL editor, create the users, profiles, niches, profile_niches, and external_links tables. Seed the niches table with your master list of 15–20 categories. Enable the PostGIS extension.
3	**Configure Supabase client**	Initialize the Supabase client in /src/lib/supabase.ts using AsyncStorage for session persistence. Wrap the app in a TanStack QueryClientProvider.
4	**Build auth screens**	Create a Welcome/Landing screen, a Sign Up screen (email + password), and a Log In screen. Use Supabase's auth.signUp() and auth.signInWithPassword(). Handle loading states and error messages.
5	**Build onboarding flow**	After sign-up, route the user through a multi-step onboarding: (1) Choose profile type: Creator or Business. (2) Enter display name and bio. (3) Select niches from the master list (multi-select chips). (4) Upload a profile picture (use Expo ImagePicker). (5) Set location (request permissions, get device coordinates, or allow manual entry). Write all data to the profiles and profile_niches tables on completion.
6	**Add navigation guards**	If the user is not logged in, show the auth stack (Welcome/SignUp/Login). If logged in but onboarding is incomplete, show the onboarding stack. If fully onboarded, show the main tab navigator.

✓ **END-OF-STAGE CHECKPOINT**

- A new user can sign up with email and password
- After sign-up, the user is guided through the full onboarding flow

- Profile data (type, name, bio, niches, picture, location) is written to Supabase
- Returning users can log in and are taken directly to the main app
- Logging out returns the user to the Welcome screen
- Navigation guards prevent unauthenticated access to the main app

STAGE 2

Account Page (Profile View & Edit)

The Account page is built early because it forces you to wire up a complete read/write loop with Supabase and produces a reusable profile card component that you'll use across the map, events, and social features. At this stage, the Account page reads from real Supabase data (written during onboarding) but everything else in the app still uses mock data.

Tasks

#	Task	Details
1	**Build profile view screen**	Display the current user's profile: profile picture (circular), display name, bio, profile type badge ('Creator' or 'Business'), years in business, niche tags, external links (rendered as tappable icons/buttons), and follower/following counts (hardcoded to 0 for now).
2	**Build profile edit screen**	An edit mode or separate screen where the user can update: display name, bio, years in business, profile picture, location, niches (add/remove from the master list), and external links (add/remove platform + URL pairs). Save changes to Supabase on submit.
3	**Create reusable ProfileCard component**	A compact card version of the profile (picture, name, niche tags, profile type) that can be used in search results, map bottom sheets, and follower lists later.
4	**Create reusable NicheTag component**	A styled chip/badge component for displaying niche labels. Used on the profile page, in filters, and on event cards.
5	**Add a Log Out button**	Place a log out option on the Account page that calls Supabase auth.signOut() and returns the user to the Welcome screen.
6	**Build 'View Other Profile' screen**	A read-only version of the profile page for viewing other users. Accepts a profile ID as a navigation param and fetches that profile from Supabase. Include a Follow/Unfollow button (non-functional for now—just UI).

✓ **END-OF-STAGE CHECKPOINT**

- Your own profile displays all data entered during onboarding
- You can edit every field and see changes persist after closing and reopening the app
- Tapping external links opens the correct URL
- The ProfileCard and NicheTag components exist and render correctly
- Log out works and returns to the Welcome screen

- The ‘View Other Profile’ screen renders when given a profile ID

STAGE 3

Creator Map

This is the app's signature feature. The map is where users discover creators, businesses, and events near them. This stage focuses on Creator Mode. Event Mode will be populated with real data in Stage 5. The map uses mock profile data for pins and will be wired to Supabase in Stage 8.

Tasks

#	Task	Details
1	**Set up react-native-maps with Mapbox**	Install and configure react-native-maps. Obtain a Mapbox API key and configure a custom map style (dark or branded theme that fits BrandMe's visual identity). Center the map on Toronto by default.
2	**Render creator pins from mock data**	Plot mock profiles as custom map markers. Use distinct pin designs for Creators vs. Businesses (different colors or icons). Implement marker clustering so the map doesn't become cluttered when zoomed out.
3	**Build bottom sheet for pin taps**	When a user taps a creator pin, a bottom sheet slides up showing the ProfileCard component (built in Stage 2). Include a 'View Full Profile' button that navigates to the 'View Other Profile' screen.
4	**Add mode toggle (Creator / Event)**	A toggle or segmented control at the top of the map to switch between Creator Mode and Event Mode. In Creator Mode, show creator/business pins. Event Mode shows placeholder pins for now (or is simply empty with a 'Coming soon' message).
5	**Add niche filter**	A horizontally scrollable row of niche chips below the mode toggle. Selecting a niche filters the visible map pins to only show profiles in that niche. Filter state should persist when switching between Creator and Event modes.
6	**Add search bar**	A search input at the top of the map that filters pins by display name. Search state should also persist across mode switches. Combine with niche filter (both active simultaneously).
7	**Handle location permissions**	Request location permissions on map load. If granted, show a 'my location' indicator on the map. If denied, default to a central Toronto view and allow manual map navigation.

✓ **END-OF-STAGE CHECKPOINT**

- The map loads centered on Toronto with a custom Mapbox style
- Mock creator and business pins are visible with distinct styling
- Tapping a pin opens a bottom sheet with the ProfileCard
- The niche filter chips filter visible pins correctly
- The search bar filters pins by name
- Creator/Event mode toggle is present and functional (Event mode is empty for now)
- Your current location dot appears on the map (if permissions granted)

STAGE 4

Events System

Events are the second core feature. This stage builds the complete events lifecycle: creating events, browsing them in a feed, RSVPing, and seeing them on the map. Events are built using mock data in the feed but the Create Event flow writes to Supabase so you can start testing real data creation.

Tasks

#	Task	Details
1	**Create events & event_rsvps tables**	In Supabase, create the events table (id, host_id, title, description, event_type enum, start_time, end_time, latitude, longitude, address_name, status with values DRAFT/PUBLISHED/CANCELLED, created_at) and the event_rsvps table (event_id, profile_id, status enum INTERESTED/GOING).
2	**Build Create Event screen**	A form screen accessible from the Create tab: title input, description textarea, event type picker (dropdown or segmented control for FASHION_SHOW, NETWORKING, WORKSHOP, etc.), date/time pickers for start and end, location picker (either tap a point on an embedded mini-map or search for an address using a geocoding API). Write the event to Supabase on submit.
3	**Build Home screen (Events feed)**	A vertically scrollable feed of upcoming events using mock data. Each event card shows: title, event type badge, host name and profile picture, date/time, location name, and Interested/Going counts. Cards are tappable to view full event details.
4	**Build Event Detail screen**	Full-screen event view: all event info, a mini-map showing the event location, the host's ProfileCard, a list of users who are Interested and Going (using mock data), and Interested/Going action buttons.
5	**Implement RSVP buttons**	Interested and Going buttons on event cards and the detail screen. Tapping toggles the state. For now, store RSVP state locally (in-memory or AsyncStorage). This will be wired to the event_rsvps table in Stage 8.
6	**Add niche filter to feed**	The same horizontally scrollable niche chip row used on the map. Filtering the feed by niche should filter events based on the host's niches or an event's associated niche.
7	**Populate Event Mode on the map**	Switch to Event Mode on the map to see mock events as pins. Use a different pin style than creators (e.g., calendar

		icon). Tapping an event pin opens a bottom sheet with event summary info and a ‘View Details’ button.

✓ END-OF-STAGE CHECKPOINT

- You can create an event and it’s written to Supabase successfully
- The Home screen shows a scrollable feed of mock events with styled cards
- Tapping an event card opens the full Event Detail screen
- Interested and Going buttons toggle visually and store state locally
- The niche filter works on the events feed
- Event Mode on the map shows mock event pins with tappable bottom sheets

STAGE 5

My Events & Event Management

This stage completes the events experience from the user's perspective. Users can see events they've expressed interest in, events they're attending, and manage events they've hosted.

Tasks

#	Task	Details
1	**Build My Events screen**	Two tabs or a segmented control: Interested and Going. Each tab shows a list of event cards the user has RSVPed to (filtered by RSVP status). Since RSVP data is still stored locally at this point, pull from local state.
2	**Add 'My Hosted Events' section**	On the Account page, add a section showing events the current user has created. Each card shows event title, date, and RSVP counts. Tapping opens the Event Detail screen.
3	**Build event edit functionality**	The host can tap into their own event and edit details (title, description, time, location). Include a 'Cancel Event' option that sets the event status to CANCELLED and visually marks it in the feed.
4	**Add empty states**	Handle cases where the user has no events in Interested, Going, or Hosted tabs. Show friendly empty state messages with an illustration or prompt (e.g., 'No events yet—explore the map to find what's happening!').

✓ **END-OF-STAGE CHECKPOINT**

- The My Events screen shows separate Interested and Going lists
- The Account page shows events you've created with correct RSVP counts
- You can edit your own events and see changes reflected
- Cancelling an event marks it as cancelled in the feed
- Empty states display correctly when there are no events in a category

STAGE 6

Social Graph & Posts

This stage adds the social layer: following other users, viewing follower/following lists, and the content posting system. It also introduces Supabase Storage for media uploads.

Tasks

#	Task	Details
1	**Create follows & posts tables**	In Supabase, create the follows table (follower_id, following_id, created_at) and the posts table (id, author_id, media_url, caption, created_at). Add follower_count and following_count as columns on the profiles table for denormalized counts.
2	**Implement follow/unfollow**	Add a Follow/Unfollow button to the 'View Other Profile' screen. Tapping writes to the follows table and updates both users' denormalized counts. The button state should reflect the current follow relationship.
3	**Build follower/following lists**	On the Account page, tapping the follower or following count opens a list of ProfileCards. Each card is tappable to navigate to that user's profile.
4	**Set up Supabase Storage**	Create a storage bucket for user media (post images/videos and profile pictures). Configure access policies so users can upload to their own folder and all users can read.
5	**Build post creation flow**	On the Create screen, add the ability to create a content post (in addition to events). The user selects an image or video from their camera roll (Expo ImagePicker), writes a caption, and submits. The media is uploaded to Supabase Storage and the post record is written to the posts table.
6	**Add posts feed to Home screen**	Add a tab or toggle on the Home screen to switch between Events feed and Posts feed. The Posts feed is a vertically scrollable list of post cards showing the author's profile picture, name, media, caption, and timestamp.
7	**Add portfolio section to profiles**	Create a portfolio_images table (profile_id, image_url, display_order). On the Account page and View Other Profile screen, show a grid of portfolio images below the bio. Allow the profile owner to add/remove/reorder images.

✓ **END-OF-STAGE CHECKPOINT**

- You can follow and unfollow other users from their profile
- Follower and following counts update in real-time on profiles

- Tapping follower/following counts shows a list of users
- You can create a post with an image/video and caption
- The Home screen has both Events and Posts feeds
- Profile pages show a portfolio image grid
- Media uploads work via Supabase Storage

STAGE 7

Go Live — Replace Mock Data with Supabase

The UI is now stable. This stage is a dedicated wiring pass where you replace all remaining mock data with real Supabase queries. Because your TypeScript types already mirror the schema, this should be a methodical process rather than a rewrite.

Tasks

#	Task	Details
1	**Wire up the Creator Map**	Replace mock profile pins with a Supabase query. Use PostGIS's ST_DWithin function (via Supabase's .rpc() method) to fetch profiles near the user's current location. Only fetch profiles within the visible map bounds to avoid loading the entire database.
2	**Wire up the Events feed**	Replace mock event data with a Supabase query: select upcoming events ordered by start_time, joined with host profile data. Apply niche filters as query parameters.
3	**Wire up RSVPs**	Move RSVP state from local storage to the event_rsvps table. The Interested/Going buttons should now write to and read from Supabase. The My Events screen should query event_rsvps filtered by the current user.
4	**Wire up Event Mode on the map**	Event pins on the map now come from a Supabase query using PostGIS, filtered by the visible map region and optionally by niche.
5	**Enable Row Level Security (RLS)**	Write RLS policies for every table. Users can read all profiles and events but can only update their own. Users can only insert/delete their own RSVPs and follows. Posts can be read by anyone but only created/deleted by the author. Test every policy to make sure unauthorized access is blocked.
6	**Add TanStack Query caching**	Wrap all Supabase calls in TanStack Query hooks (useQuery for reads, useMutation for writes). Configure stale times appropriate for each data type: profiles can be cached longer, events and RSVPs should refresh more frequently.
7	**Handle loading and error states**	Add loading skeletons or spinners for every screen that fetches data. Add error boundaries and retry logic for failed queries. Add pull-to-refresh on the Events feed, Posts feed, and My Events screen.

✓ END-OF-STAGE CHECKPOINT

- Every screen in the app reads from and writes to Supabase (no mock data remains)
- The map loads real profiles and events based on geographic proximity
- RSVPs persist in the database and appear on My Events
- RLS policies are tested and working (unauthorized writes are rejected)
- Loading spinners, skeletons, and error states are visible throughout the app
- Pull-to-refresh works on all list screens

STAGE 8

Polish, Search & Discovery

With all core features working on real data, this stage focuses on the quality and discoverability that make the app feel finished. This is where you go from “it works” to “it feels good to use.”

Tasks

#	Task	Details
1	**Build global search**	A dedicated search screen or overlay accessible from the Home screen. Search across creators (by name, niche, bio keywords) and events (by title, description, location). Return results in two sections: People and Events. Each result uses the ProfileCard or event card components.
2	**Refine map interactions**	Add smooth map animations when selecting a pin. Improve the bottom sheet to support swipe-to-dismiss and swipe-to-expand (showing more details). Add a ‘recenter on me’ button. Ensure pin clustering animates when zooming in/out.
3	**Visual polish pass**	Consistent spacing, typography, and color usage across all screens. Smooth transitions between screens. Ensure the app looks sharp on both iOS and Android. Test on multiple screen sizes.
4	**Add push notifications**	Integrate Expo Notifications. Notify users when: someone follows them, an event they’re interested in is starting soon (24-hour reminder), or a new event is posted in a niche they follow. Register the push token with Supabase.
5	**Implement image caching**	Use expo-image or react-native-fast-image for efficient image loading and caching. Profile pictures, post images, and event images should load once and cache locally.
6	**Accessibility pass**	Ensure all interactive elements have accessibility labels. Test with VoiceOver (iOS) and TalkBack (Android). Ensure sufficient color contrast on all text.

✓ **END-OF-STAGE CHECKPOINT**

- Global search returns relevant creators and events quickly
- Map interactions feel smooth with animations and expandable bottom sheets
- The app looks visually polished and consistent across all screens
- Push notifications are delivered for follows and event reminders
- Images load fast and don’t re-download on revisit
- The app is usable with screen readers

STAGE 9

Analytics, Testing & Launch Prep

The final stage prepares BrandMe for real users. You'll instrument the app with analytics, build it for the app stores, and run your first round of community testing in Toronto.

Tasks

#	Task	Details
1	**Integrate PostHog**	Install the PostHog React Native SDK. Track key events: screen views, profile views, event RSVPs, follows, post creation, map interactions (zoom, pin taps, mode switches), search queries, and onboarding completion rate.
2	**Set up EAS builds**	Configure eas.json for development, preview, and production build profiles. Run a preview build and install on your device outside of Expo Go to test the full native experience (push notifications, maps, image picker).
3	**Create App Store assets**	Design the app icon, splash screen, and App Store/Play Store screenshots. Write the app description and metadata. Set up the App Store Connect and Google Play Console listings.
4	**Seed real data for testing**	Create 20–30 realistic test profiles across different niches and locations in Toronto. Create 10–15 upcoming events. This data should look real enough that test users feel like they're using an active platform.
5	**Run community test**	Distribute the preview build to 10–20 Toronto-based creators via TestFlight and Android internal testing. Gather feedback on: onboarding friction, map usability, event discovery, overall value proposition. Monitor PostHog for usage patterns and drop-off points.
6	**Bug fix and iterate**	Address critical bugs and UX friction surfaced during testing. Prioritize issues that prevent core actions (signing up, finding creators, RSVPing to events) over cosmetic polish.

✓ **END-OF-STAGE CHECKPOINT**

- PostHog dashboard shows event tracking for all core user actions
- The app runs as a standalone build (not through Expo Go) on iOS and Android
- App Store assets and metadata are ready for submission
- Test data makes the app feel populated and active
- Community testers have provided feedback

- Critical bugs from testing are resolved
- The app is ready for its first public release in Toronto

Appendix: Database Schema Reference

This is the complete set of tables needed for BrandMe, including additions recommended beyond the initial schema. Tables are listed in the order they should be created (respecting foreign key dependencies).

users

Core authentication table. Managed by Supabase Auth, so you typically don't create this manually—Supabase creates it in the auth schema. Your profiles table references auth.users.id.

profiles

The public-facing user identity. Fields: id (UUID, PK), user_id (FK to auth.users.id), display_name, bio, profile_type ('CREATOR' | 'BUSINESS'), years_in_business (integer), profile_picture_url, latitude (float), longitude (float), follower_count (integer, default 0), following_count (integer, default 0), created_at. The denormalized follower/following counts avoid expensive JOIN queries when loading profiles.

niches

Master list of categories. Fields: id (PK), name (unique). Seeded once during setup with entries like Streetwear, Cinematography, Barbering, Music Production, etc.

profile_niches

Many-to-many link between profiles and niches. Fields: profile_id (FK), niche_id (FK). Composite primary key on both columns.

external_links

Social handles and website URLs. Fields: id (PK), profile_id (FK), platform_name (e.g., 'Instagram', 'TikTok', 'YouTube'), url.

events

All community events. Fields: id (UUID, PK), host_id (FK to profiles.id), title, description, event_type ('FASHION_SHOW' | 'NETWORKING' | 'WORKSHOP' | 'LISTENING_PARTY' | 'COMMUNITY' | 'OTHER'), start_time (timestamp), end_time (timestamp), latitude, longitude, address_name, status ('DRAFT' | 'PUBLISHED' | 'CANCELLED'), created_at.

event_rsvps

Tracks interest and attendance. Fields: event_id (FK), profile_id (FK), status ('INTERESTED' | 'GOING'). Composite primary key on event_id + profile_id.

follows

Social graph edges. Fields: follower_id (FK to profiles.id), following_id (FK to profiles.id), created_at. Composite primary key on both IDs. A database trigger should increment/decrement the denormalized counts on profiles.

posts

User-generated content. Fields: id (PK), author_id (FK to profiles.id), media_url (string pointing to Supabase Storage), caption (text), created_at.

portfolio_images

Showcase images on profiles. Fields: id (PK), profile_id (FK), image_url (string pointing to Supabase Storage), display_order (integer), created_at. An addition to the original schema—allows creators to highlight their best work beyond a single profile picture.